When the macaroon is presented back to the API, it can use the original HMAC key to reconstruct the original tag and all the caveat tags and check if it comes up with the same signature value at the end of the chain of caveats. Listing 9.14 shows an example of how to verify an HMAC chain just like that used by macaroons.

First initialize a `javax.crypto.Mac` object with the API's authentication key (see chapter 5 for how to generate this) and then compute an initial tag over the macaroon unique identifier. You then loop through each caveat in the chain and compute a new HMAC tag over the caveat, using the old tag as the key[39]. Finally, you compare the computed tag with the tag that was supplied with the macaroon using a constant-time equality function. Listing 9.14 is just to demonstrate how it works, you'll use a real macaroon library in the Natter API, so you don't need to implement this method.

Listing 9.14 Verifying the HMAC chain

```
private boolean verify(String id, List<String> caveats,
byte[] tag)
throws Exception {
var hmac = Mac.getInstance("HmacSHA256"); #A
hmac.init(macKey); #A
var computed = hmac.doFinal(id.getBytes(UTF_8)); #B
for (var caveat : caveats) { #C
hmac.init(new SecretKeySpec(computed, "HmacSHA256")); #C
computed = hmac.doFinal(caveat.getBytes(UTF_8)); #C
}
return MessageDigest.isEqual(tag, computed); #D
}
```